Lab Manual

Data structures lab

(Recursion)

Lab Instructor: Muhammad Moiz Khan

Objectives:

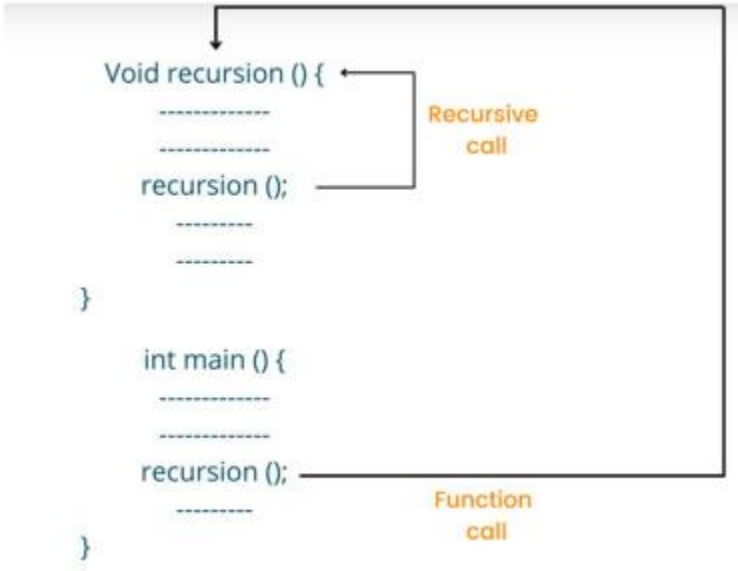
After completing this lab, students will be able to:

- Understand the concept of recursion.
- Differentiate between recursion and iteration.
- Implement basic recursive functions in C++.
- Trace recursive calls using recursion tree and stack concept.
- Solve real-life problems using recursion.
- Analyze time and space complexity of recursive solutions.

Recursion:

- The process in which a function calls itself directly or indirectly is called recursion
- The corresponding function is called a recursive function.
- A recursive function solves a particular problem by calling a copy of itself
- Recursion has the ability of solving smaller subproblems of the original problems
- Many more recursive calls can be generated as and when required.
- It is essential to know that we should provide a certain case in order to terminate this recursion process
- So we can say that every time the function calls itself with a simpler version of the original problem

How Recursion works:



Properties of recursion:

- Recursion is an amazing technique with the help of which we can reduce the length of our code and make it easier to read and write.
- Performing the same operations multiple times with different inputs
- In every step, we try smaller inputs to make the problem smaller
- Base condition is needed to stop the recursion otherwise infinite loop will occur

Advantages of recursion:

- Less number code lines are used in the recursion program and hence the code looks shorter and cleaner.
- Recursion is easy to approach to solve the problems involving data structure and algorithms like graph and tree
- Recursion helps to reduce the time complexity
- It helps to reduce unnecessary calling of the function
- It helps to solve the stack evolutions and prefix, infix, postfix evaluation
- Recursion is the best method to define objects that have repeated structural forms

Disadvantages of recursion:

- It consumes a lot of stack space
- It takes more time to process the program
- If an error is accrued in the program, it is difficult to debug the error in comparison to the iterative program.

Recursion vs iteration:

SR No.	Recursion	Iteration
1)	Terminates when the base case becomes true.	Terminates when the condition becomes false.
2)	Used with functions.	Used with loops.
3)	Every recursive call needs extra space in the stack memory.	Every iteration does not require any extra space.
4)	Smaller code size.	Larger code size.

Recursion function stored in memory as:

- Recursion uses more memory, because the recursive function adds to the stack with each recursive call
- It keeps the values there until the call is finished
- The recursive function uses LIFO (LAST IN FIRST OUT) Structure just like the stack data structure

Basics of recursion:

- There are two cases in the recursion/ recursive programs:
  ➢Base case
  ➢Recursive case
- In the recursive program, the solution to the base case is provided
- Base case terminates the recursive function
- Solution to the bigger problem is expressed in terms of smaller problems

```
return_type function_name(parameters) {
    if (base_condition)
        return base_value;
    else
        return function_name(smaller_input);
}
```

Example Problems:

Problem 1: Write a recursive function to calculate factorial of a number.

Formula:
n! = n × (n-1)!
Base case: 0! = 1

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    cout << "Factorial = " << factorial(num);
    return 0;
}
```

Problem 2: Print nth Fibonacci number using recursion.

Formula:
F(n) = F(n-1) + F(n-2)
Base cases:
F(0) = 0
F(1) = 1

```
#include <iostream>
using namespace std;

int fibonacci(int n) {
    if (n == 0)
        return 0;
    if (n == 1)
        return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n;
    cout << "Enter position: ";
    cin >> n;

    cout << "Fibonacci number = " << fibonacci(n);
    return 0;
}
```

Problem 3: Sum of First N Natural Numbers

```
#include <iostream>
using namespace std;

int sum(int n) {
    if (n == 0)
        return 0;
    return n + sum(n - 1);
}

int main() {
    int n;
    cout << "Enter n: ";
    cin >> n;

    cout << "Sum = " << sum(n);
    return 0;
}
```

Lab Tasks:

Task 1. Reverse a String Using Recursion

Task 2. Find product of two numbers using recursive addition.

Task 3. Count digits in a number recursively.

Task 4. Find sum of digits of a number.